

Classical ML to NNS



Supervised ML

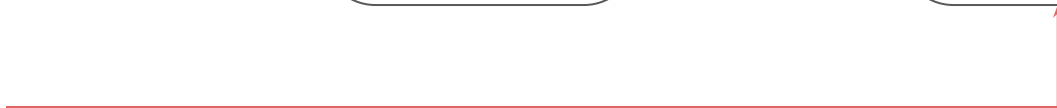


AI
Model

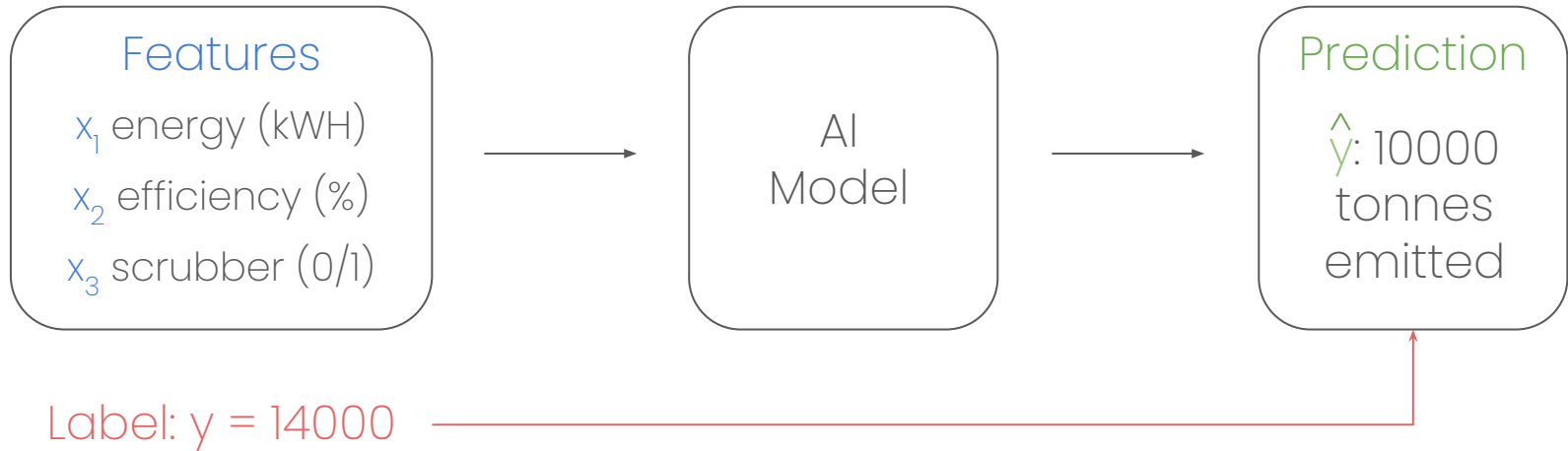


Prediction:
Lynx

Label: Cat



Ex: Power Plant CO₂ Emissions

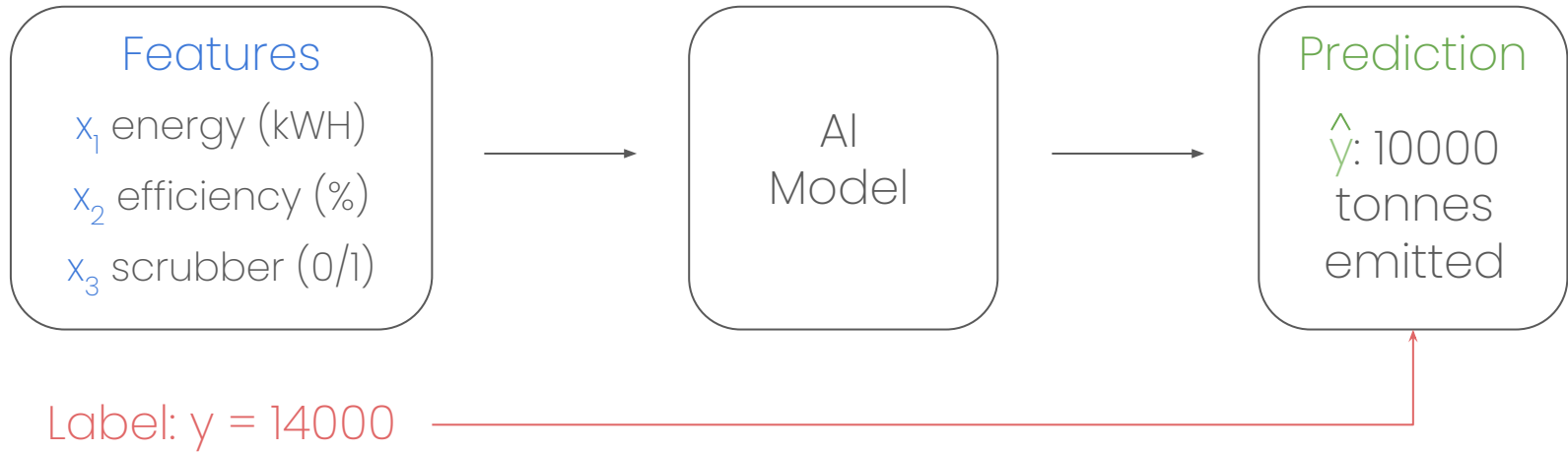


3 Parts of Supervised ML

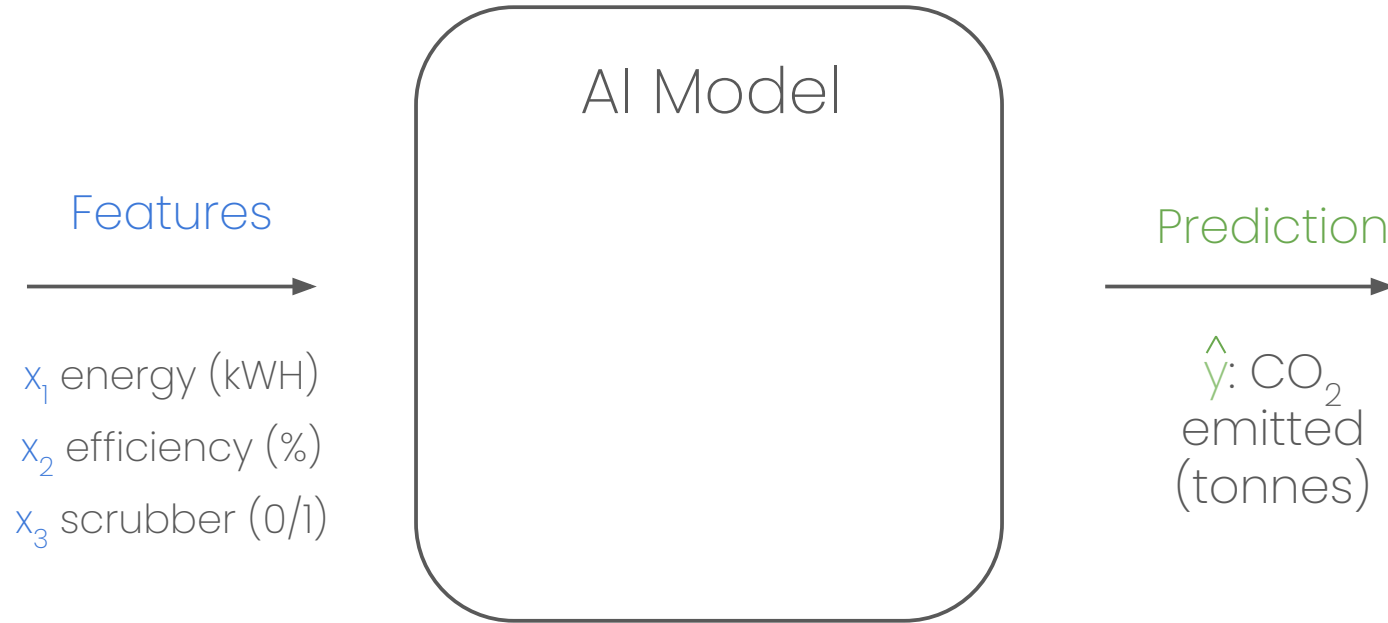
(Predictions, Error, Optimisation)



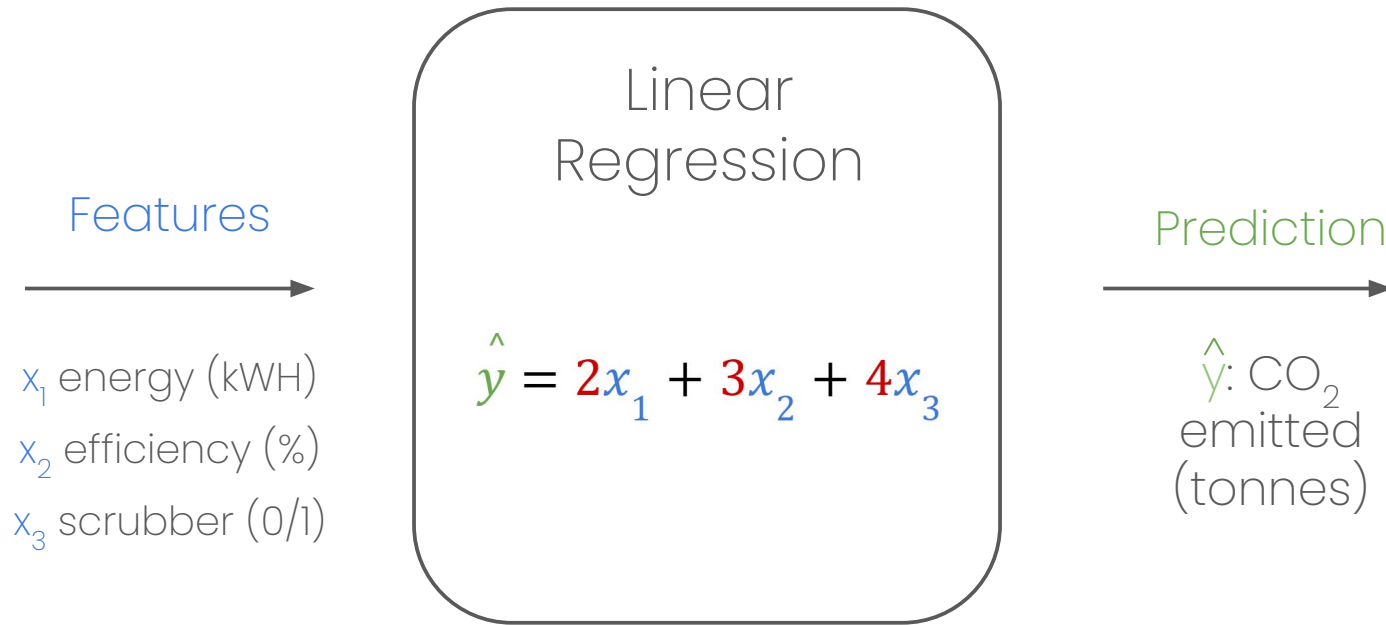
Creating Predictions



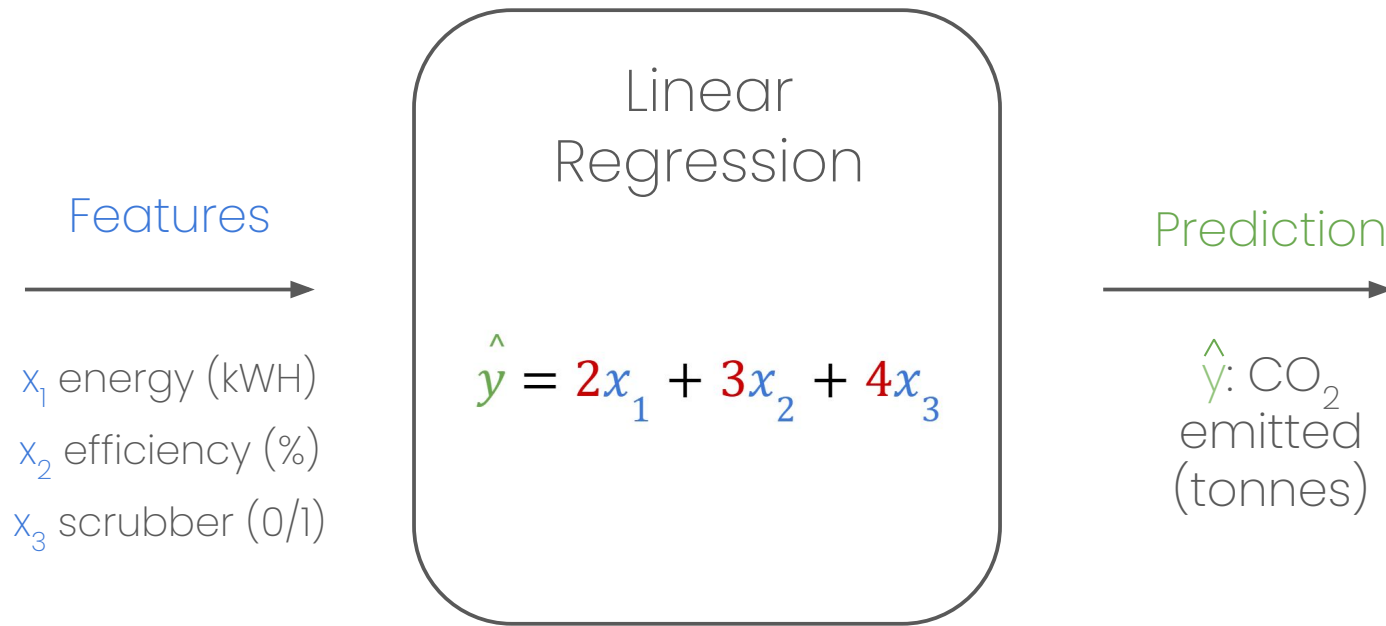
Creating Predictions



Problem solved?



Nope: how to pick parameters?

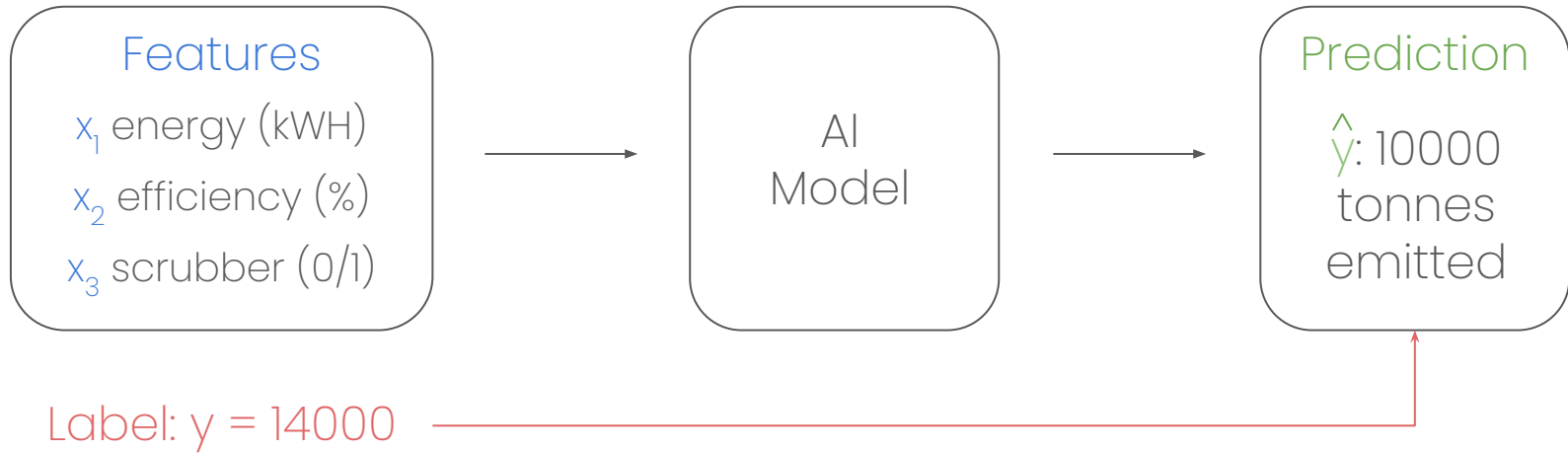


3 Parts of Supervised ML

(Predictions, Error, Optimisation)



How to measure error?



Idea 1: Difference

$$y - \hat{y}$$

$$14000 - 10000$$

$$4000$$



Idea 2: Total Difference

$$(y^{(1)} - \hat{y}^{(1)}) + (y^{(2)} - \hat{y}^{(2)})$$

$$(14000 - 10000) + (8000 - 12000)$$

$$0$$



Idea 2: Total Squared Difference

$$(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2$$

$$(14000 - 10000)^2 + (8000 - 12000)^2$$

$$32,000,000$$



Idea 3: Mean Squared Difference

$$[(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + \dots + (y^{(N)} - \hat{y}^{(N)})^2]/N$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$



3 Parts of Supervised ML

(Predictions, Error, Optimisation)

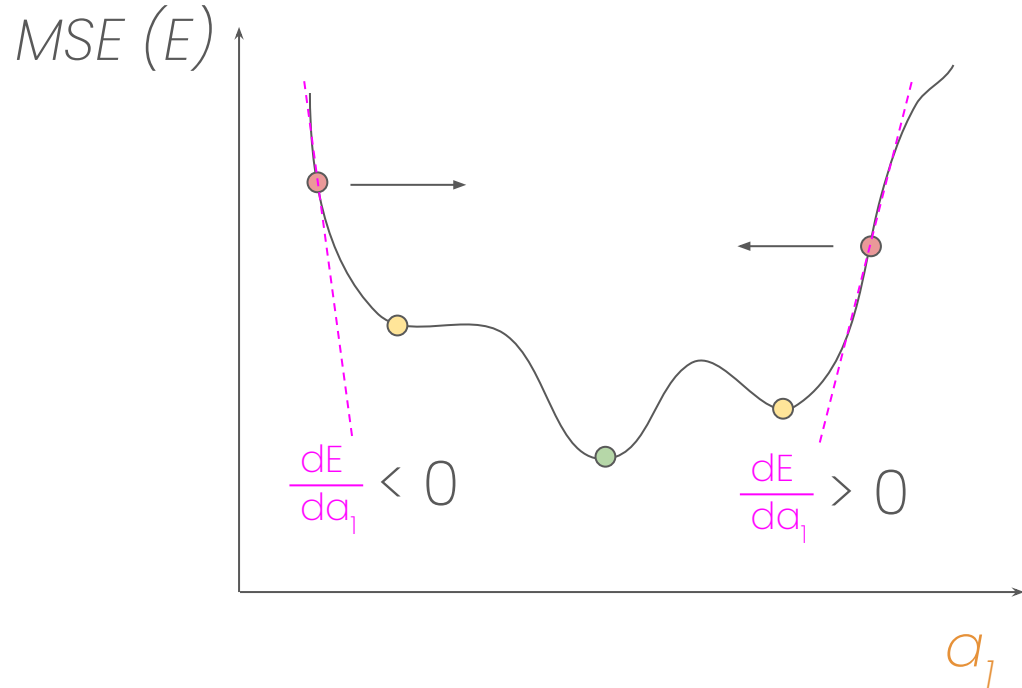


Gradient Descent

$$\hat{y} = a_1 x_1$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

$$a_1 = a_1 - \frac{dE}{da_1}$$

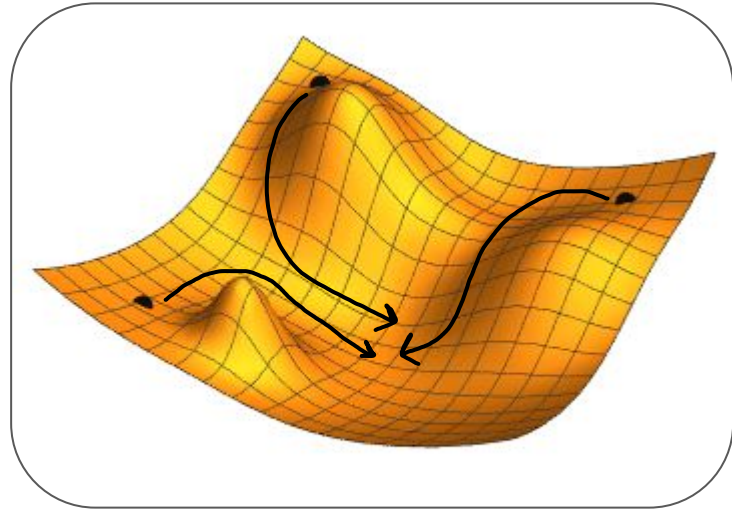


Gradient Descent

$$\hat{y} = a_1 x_1 + a_2 x_2 + \dots$$

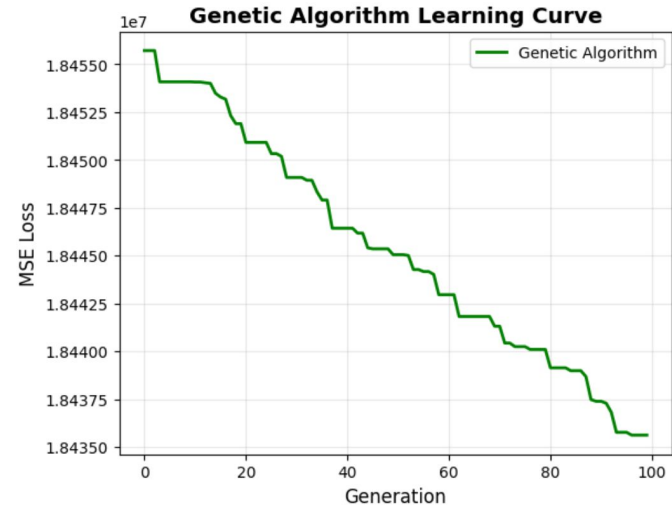
$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

$$\vec{a} = \vec{a} - \alpha \nabla E(\vec{a})$$



Genetic Algorithm (Smart Trial & Error)

1. Make 100 sets of random parameters ($a_1, a_2, a_3, \dots$)
2. Check error for each set
3. Worst 5 sets replaced with new random sets.
4. Other sets get more 'mutations' with higher error. Ex: $\vec{a} = \vec{a} + E \cdot \text{rand}$.





3 Parts of Supervised ML

(Predictions, Error, Optimisation)

(Except More Complicated)



Logistic Regression

Linear
Regression

$$\hat{y} = 2x_1 + 3x_2 + 4x_3$$

Logistic
Regression

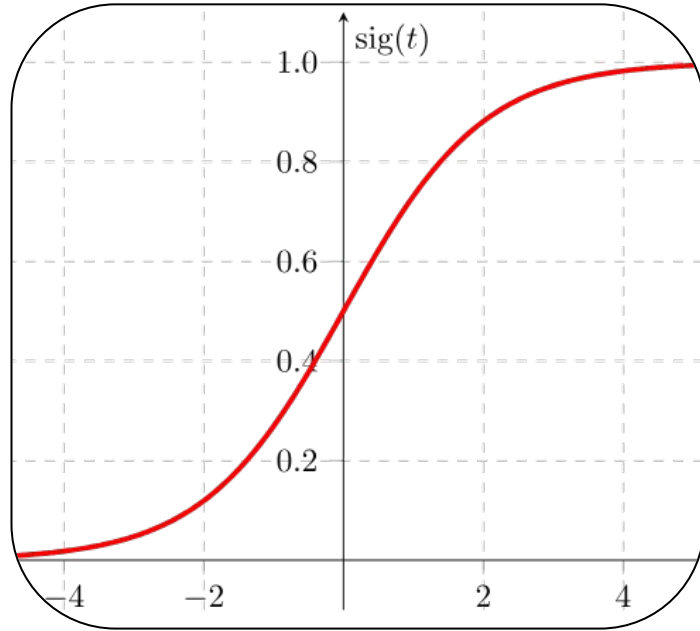
$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Logistic Regression (probabilities)



Logistic
Regression

$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Sidenote: Binary Cross Entropy

MSE
(Big Numbers)

$$MSE = \frac{1}{N} \sum_{k=1}^N (\mathbf{y}^{(k)} - \mathbf{y}^{(k)})^2$$

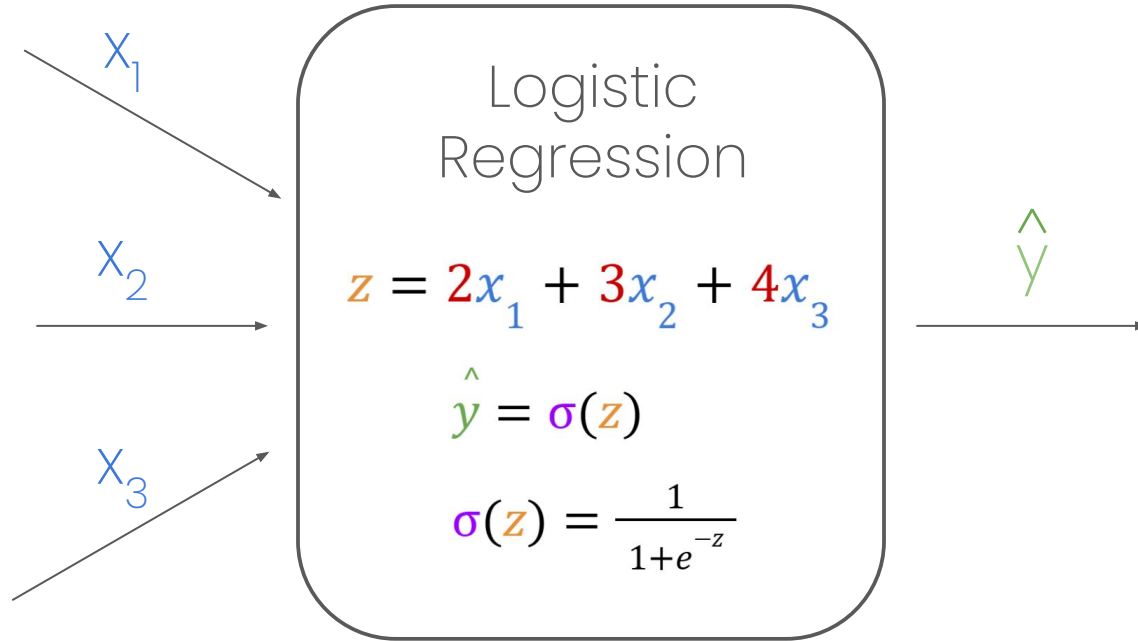
BCE
(Probabilities)

$$BCE = \frac{1}{N} \sum_{k=1}^N [BCE_k]$$

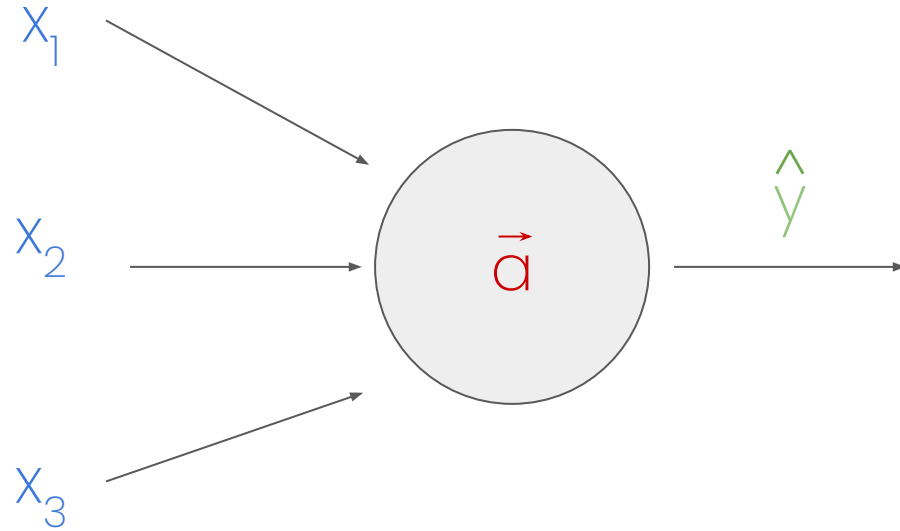
$$BCE_k = \mathbf{y}^{(k)} \log(\mathbf{y}^{(k)}) \\ + (1 - \mathbf{y}^{(k)}) \log(1 - \mathbf{y}^{(k)})$$



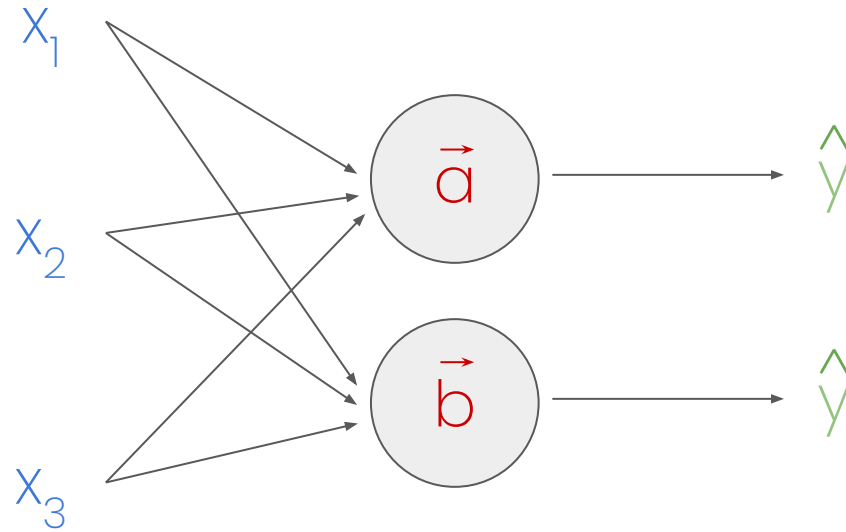
Perceptron



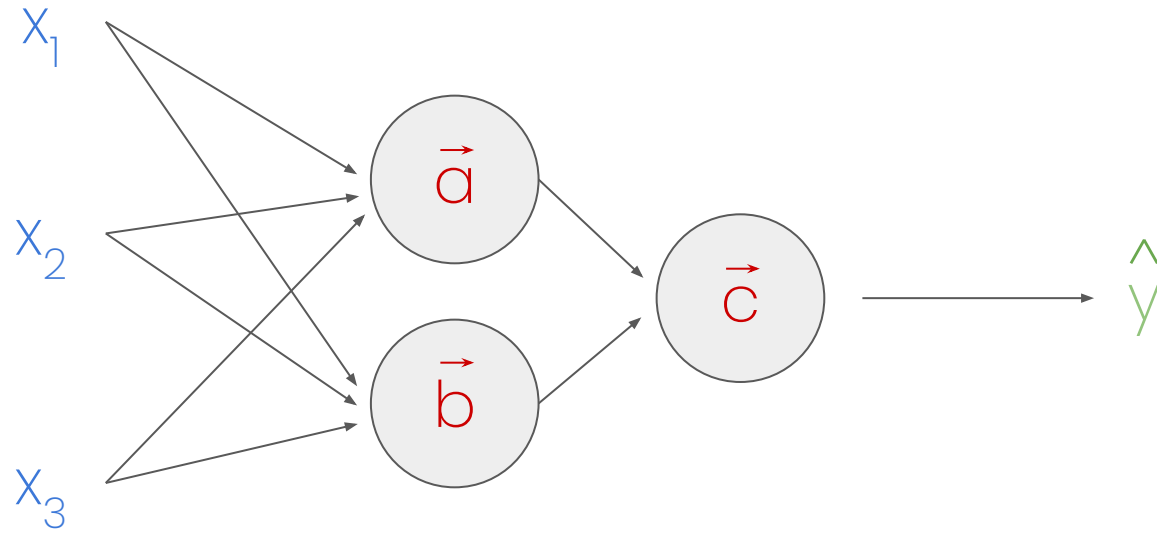
Multi-layer Perceptron



Multi-layer Perceptron

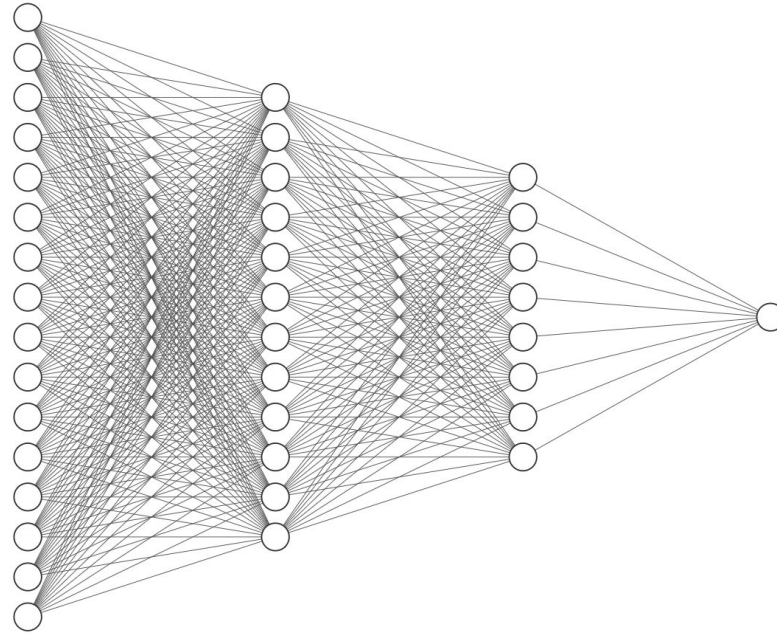


Multi-layer Perceptron



Neural Network

Inputs



Prediction

Hidden layers



Activation Functions

Replace **sigmoid**
as needed

Neuron

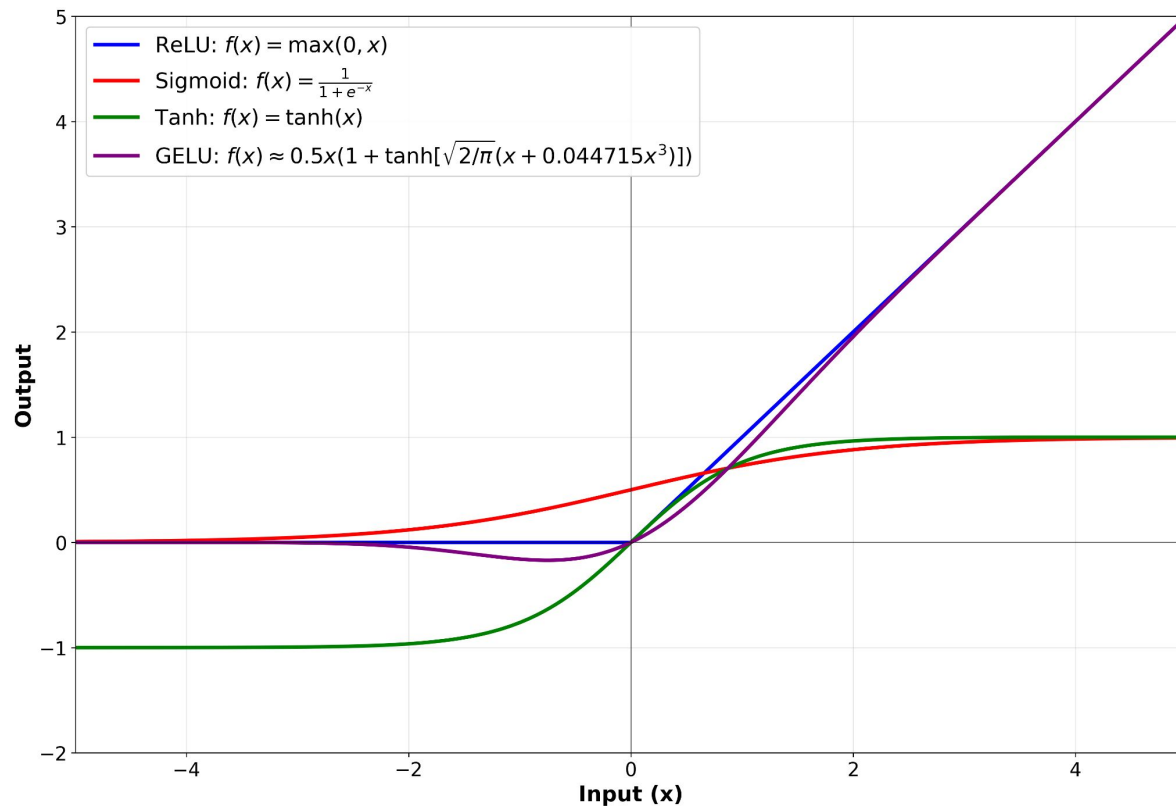
$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Activation Functions



So just add fancy equations, no worries?

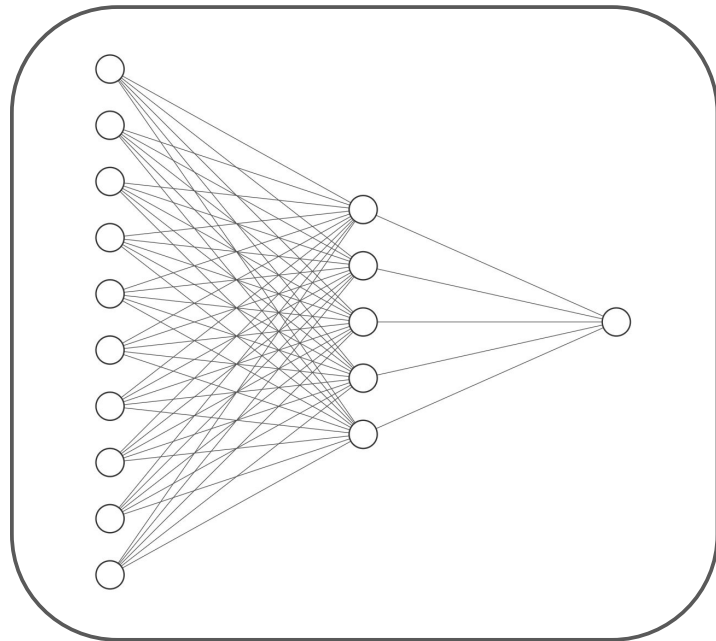


Gradient Descent (Revisited)

$$\hat{y} = a_1 x_1 + a_2 x_2 + \dots$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

$$\vec{a} = \vec{a} - \alpha \nabla E(a)$$



Gradient Descent (New Equations)

$$\vec{z}_1 = W_1 \vec{x}_1 + \vec{b}_1 \quad - \vec{x}_1, \vec{b}_1 \in \mathbb{R}^{10}, W_1 \in \mathbb{R}^{5 \times 10}$$

$$\vec{a}_1 = \sigma(\vec{z}_1) \quad - \vec{a}_1, \vec{z}_1 \in \mathbb{R}^5$$

$$\vec{z}_2 = W_2 \vec{a}_1 + \vec{b}_2 \quad - \vec{a}_1, \vec{b}_2 \in \mathbb{R}^5, W_2 \in \mathbb{R}^{1 \times 5}$$

$$\hat{y} = \sigma(\vec{z}_2) \quad - \hat{y}, \vec{z}_2 \in \mathbb{R}$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$



Gradient Descent (New Equations)

$$\vec{z}_1 = W_1 \vec{x}_1 + \vec{b}_1 \quad - \vec{x}_1, \vec{b}_1 \in \mathbb{R}^{10}, W_1 \in \mathbb{R}^{5 \times 10}$$

$$\vec{a}_1 = \sigma(\vec{z}_1) \quad - \vec{a}_1, \vec{z}_1 \in \mathbb{R}^5$$

$$\vec{z}_2 = W_2 \vec{a}_1 + \vec{b}_2 \quad - \vec{a}_1, \vec{b}_2 \in \mathbb{R}^5, W_2 \in \mathbb{R}^{1 \times 5}$$

$$\hat{y} = \sigma(\vec{z}_2) \quad - \hat{y}, \vec{z}_2 \in \mathbb{R}$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

Forward Pass

$$W_1 = W_1 - \alpha \nabla E(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(\hat{y})$$

$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \sigma'(\vec{z}_2) \cdot \nabla \vec{z}_2(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(\hat{y}) \cdot \sigma'(\vec{z}_2) \cdot \nabla \vec{z}_2(\vec{a}_1) \cdot \dots$$

Backpropagation



~~Gradient Descent~~ Chain Rule Round ∞

$$\vec{z}_1 = W_1 \vec{x}_1 + \vec{b}_1 \quad - \vec{x}_1, \vec{b}_1 \in \mathbb{R}^{10}, W_1 \in \mathbb{R}^{5 \times 10}$$

$$\vec{a}_1 = \sigma(\vec{z}_1) \quad - \vec{a}_1, \vec{z}_1 \in \mathbb{R}^5$$

$$\vec{z}_2 = W_2 \vec{a}_1 + \vec{b}_2 \quad - \vec{a}_1, \vec{b}_2 \in \mathbb{R}^5, W_2 \in \mathbb{R}^{1 \times 5}$$

$$\hat{y} = \sigma(\vec{z}_2) \quad - \hat{y}, \vec{z}_2 \in \mathbb{R}$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

Forward Pass

$$W_1 = W_1 - \alpha \nabla E(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(\hat{y})$$

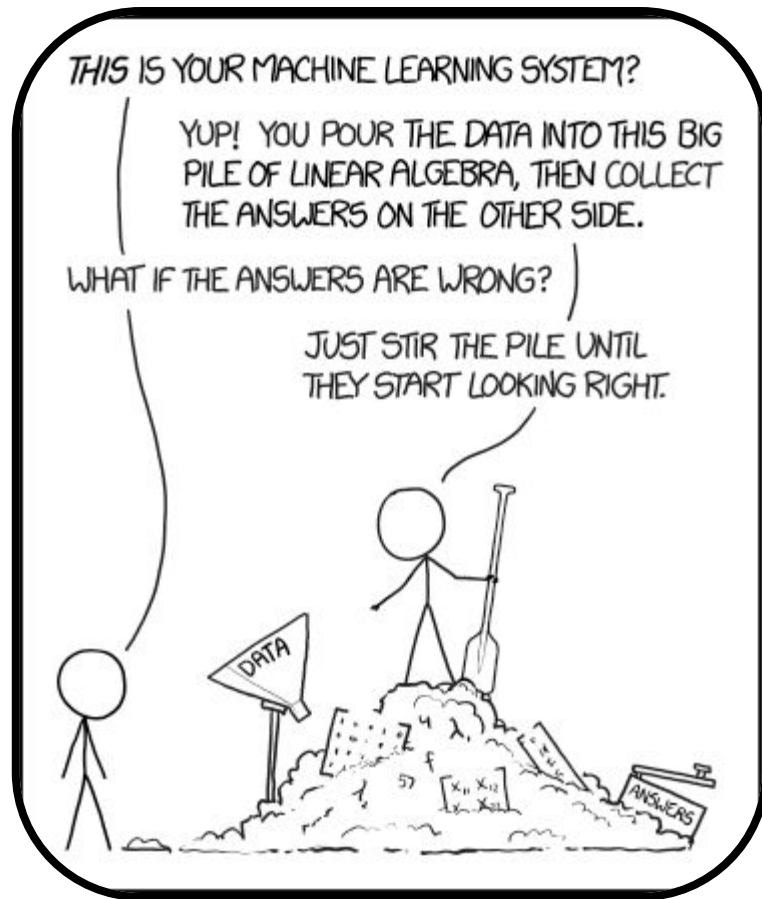
$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \sigma'(\vec{z}_2) \cdot \nabla \vec{z}_2(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(\hat{y}) \cdot \sigma'(\vec{z}_2) \cdot \nabla \vec{z}_2(\vec{a}_1) \cdot \dots$$

Backpropagation



[xkcd #1838](#)





Demo 2

